



UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

Università degli Studi di Napoli - Corso di Laurea in Ingegneria informatica

Salvatore Vinciguerra N46007541

Matteo Fruggiero N46007530

Nicola Gesumaria N46007590

1. Obiettivo del Progetto.....	3
2. Specifiche del dataset.....	4
3. Fondamenti teorici e metriche di ricerca.....	4
4. Algoritmi di ricerca non informata.....	6
4.1 Breadth-First Search.....	6
4.2 Depth-First Search (DFS).....	7
4.3 Depth-Limited Search (DLS).....	7
4.4 Iterative Deepening Search (IDS).....	8
5. Implementazione Python.....	9
6. Workflow KNIME.....	10
7. Analisi dei Risultati Sperimentali e Reportistica Grafica.....	13
8. Conclusioni e Linee Guida per la Scelta dell'Algoritmo Ottimale.....	15

1. Obiettivo del Progetto

L'obiettivo del progetto è sviluppare e testare un software per la ricerca di percorsi all'interno di uno spazio degli stati rappresentato tramite grafo. Dato un dataset contenente le connessioni tra gli stati, il sistema deve individuare, se esiste, una sequenza di nodi che permetta di raggiungere uno stato obiettivo, detto **goal**, partendo da uno **stato iniziale** scelto dall'utente. Il programma implementa e confronta quattro algoritmi di ricerca non informata:

- **Breadth-First Search (BFS)**: ricerca in ampiezza, che esplora prima i nodi più vicini allo stato iniziale;
- **Depth-First Search (DFS)**: ricerca in profondità, che segue un ramo del grafo fino al suo esaurimento prima di tornare indietro;
- **Depth-Limited Search (DLS)**: variante della DFS in cui viene imposto un limite massimo di profondità;
- **Iterative Deepening Search (IDS)**: strategia che applica più volte la DLS aumentando progressivamente il limite di profondità.

Il problema viene considerato in un contesto deterministico e completamente osservabile. Gli archi del grafo sono trattati come non pesati, quindi ogni transizione ha costo uniforme. Di conseguenza, la qualità della soluzione viene valutata in base al numero di archi attraversati per raggiungere il goal.

Oltre alla correttezza del percorso individuato, il progetto analizza anche le prestazioni degli algoritmi in termini di **tempo di esecuzione**, **numero di iterazioni**, **lunghezza del percorso** e **memoria utilizzata**. I risultati prodotti dallo script Python vengono poi importati in **KNIME**, dove sono organizzati, confrontati e rappresentati tramite tabelle e grafici.

2. Specifiche del dataset

Il programma utilizza due **file CSV** distinti: un file contenente i nodi e un file contenente gli archi. Il file dei nodi associa a ogni identificativo una label, mentre il file degli archi descrive le connessioni tra i nodi del grafo.

Durante il caricamento, il software legge i nodi, costruisce le corrispondenze tra ID e label e inizializza la **lista di adiacenza**. Successivamente vengono caricati gli archi: ogni riga indica una connessione tra un nodo sorgente e un nodo destinazione. Se il grafo viene impostato

come non diretto, il programma aggiunge automaticamente anche il collegamento inverso.

Il grafo viene rappresentato tramite liste di adiacenza, così da accedere rapidamente ai vicini di ogni nodo durante l'esecuzione degli algoritmi di ricerca. Prima di inserire un arco, il programma verifica che entrambi i nodi siano presenti nel file dei nodi; in caso contrario, il collegamento viene ignorato.

Il programma prevede inoltre una gestione degli errori per casi come file mancanti, nodi non trovati, parametro del grafo non valido o algoritmo non riconosciuto. In tali situazioni viene prodotto comunque un file CSV di output compatibile con KNIME, contenente un messaggio di errore.

3. Fondamenti teorici e metriche di ricerca

Nel contesto del problem solving, un problema può essere rappresentato attraverso uno spazio degli stati, cioè l'insieme di tutte le possibili configurazioni raggiungibili a partire da uno stato iniziale. Nel caso di questo progetto, lo spazio degli stati viene modellato tramite un grafo orientato $G=(V,E)$, dove l'insieme V rappresenta i nodi, cioè gli stati, mentre l'insieme E rappresenta gli archi, cioè le transizioni possibili tra uno stato e l'altro.

L'obiettivo degli algoritmi di ricerca è individuare, se esiste, un percorso che colleghi uno stato iniziale a uno stato obiettivo. Poiché il grafo considerato non è pesato, ogni arco viene trattato come avente lo stesso costo. Di conseguenza, la soluzione migliore corrisponde al percorso che raggiunge il goal attraversando il minor numero di archi.

Per confrontare gli algoritmi implementati vengono considerate alcune metriche fondamentali:

- **Completezza:** indica se l'algoritmo è in grado di trovare una soluzione quando questa esiste.
- **Complessità temporale:** misura il numero di operazioni o di nodi esplorati necessari per completare la ricerca.
- **Complessità spaziale:** indica la quantità di memoria necessaria durante l'esecuzione, in particolare per gestire strutture come frontiera, coda o pila.
- **Ottimalità:** indica se l'algoritmo restituisce sempre la soluzione migliore, cioè il percorso a costo minimo.

Nell'analisi teorica vengono utilizzati alcuni parametri ricorrenti. Il **branching factor** b indica il numero massimo di successori generabili da un nodo. La variabile d rappresenta la profondità della soluzione più vicina allo stato iniziale, mentre m indica la profondità massima dello spazio di ricerca. Questi valori permettono di stimare il comportamento degli algoritmi in termini di tempo e memoria, soprattutto quando il grafo cresce in dimensione.

Nel progetto, oltre alle proprietà teoriche, vengono considerate anche metriche sperimentali ottenute durante l'esecuzione dello script Python, come il tempo di esecuzione, il numero di iterazioni, la lunghezza del percorso trovato e la dimensione massima della struttura dati utilizzata.

4. Algoritmi di ricerca non informata

Gli algoritmi di ricerca non informata esplorano lo spazio degli stati senza utilizzare informazioni aggiuntive sulla posizione del goal. La scelta del nodo da espandere dipende quindi solo dalla strategia adottata dall'algoritmo e dalla struttura dati usata per gestire la frontiera. Nel progetto sono stati considerati quattro algoritmi: **BFS**, **DFS**, **DLS** e **IDS**.

4.1 Breadth-First Search

La **Breadth-First Search**, o ricerca in ampiezza, esplora il grafo procedendo per livelli. Prima vengono visitati tutti i nodi raggiungibili direttamente dallo stato iniziale, poi quelli a profondità 2, e così via. La frontiera viene gestita tramite una coda FIFO, in cui il primo nodo inserito è anche il primo a essere estratto.

La BFS è completa se il fattore di ramificazione b è finito. Inoltre, nel caso di grafi non pesati o con costi uniformi, garantisce il percorso ottimale, cioè quello con il minor numero di archi tra stato iniziale e goal.

Il principale limite della BFS riguarda il consumo di memoria. L'algoritmo deve infatti mantenere in memoria tutti i nodi generati ma non ancora espansi. Per questo motivo, sia la complessità temporale sia quella spaziale sono pari a:

$$O(B^{d+1})$$

dove d indica la profondità della soluzione più vicina.

Nel progetto, la BFS rappresenta l'algoritmo più adatto quando si vuole ottenere il percorso

minimo, ma può diventare onerosa su grafi molto grandi.

4.2 Depth-First Search (DFS)

La **Depth-First Search**, o ricerca in profondità, segue una strategia opposta rispetto alla BFS. L'algoritmo esplora un ramo del grafo fino alla massima profondità possibile, prima di tornare indietro e analizzare altri rami. La frontiera viene gestita tramite una struttura LIFO, cioè una pila.

La DFS ha il vantaggio di richiedere meno memoria rispetto alla BFS, poiché mantiene principalmente il cammino corrente e i nodi ancora da esplorare. La complessità spaziale è:

$$O(b \cdot m)$$

dove m rappresenta la profondità massima dello spazio di ricerca.

Dal punto di vista temporale, nel caso peggiore la DFS può dover esplorare un numero molto elevato di nodi:

$$O(b^m)$$

4.3 Depth-Limited Search (DLS)

La **Depth-Limited Search** è una variante della DFS in cui viene imposto un limite massimo di profondità l . Quando l'algoritmo raggiunge tale limite, non espande ulteriormente il nodo corrente, anche se questo possiede successori.

Questa strategia permette di evitare il problema dei rami troppo profondi o potenzialmente infiniti, ma introduce una nuova criticità: se il goal si trova a una profondità maggiore del limite impostato, l'algoritmo non riesce a trovarlo.

La DLS non è quindi completa in generale, ma può diventarlo se il limite scelto è sufficientemente grande rispetto alla profondità della soluzione. La complessità temporale è:

$$O(b^l)$$

mentre quella spaziale è:

$$O(b \cdot l)$$

4.4 Iterative Deepening Search (IDS)

La **Iterative Deepening Search** combina alcuni vantaggi della BFS e della DFS. L'algoritmo esegue più volte una Depth-Limited Search, aumentando progressivamente il limite di profondità. Si parte da $l=0$, poi $l=1$, $l=2$, fino a quando il goal viene trovato.

Anche se i nodi dei livelli superiori vengono rigenerati più volte, questo costo aggiuntivo è generalmente contenuto, perché la maggior parte dei nodi si trova nei livelli più profondi dell'albero di ricerca.

La IDS è completa se il fattore di ramificazione è finito. Inoltre, in grafi non pesati o con costi uniformi, trova una soluzione ottimale come la BFS, ma con un consumo di memoria più vicino a quello della DFS.

La complessità temporale è:

$$O(b^d)$$

mentre la complessità spaziale è:

$$O(b \cdot d)$$

Nel progetto, la IDS rappresenta un buon compromesso: mantiene buone garanzie sulla soluzione trovata e riduce il consumo di memoria rispetto alla BFS.

5. Implementazione Python

L'implementazione Python costituisce il **modulo computazionale** del progetto. Il programma ha il compito di leggere i dati relativi al grafo, costruire una rappresentazione interna efficiente e applicare uno degli algoritmi di ricerca non informata previsti: BFS, DFS, DLS e IDS. Lo script gestisce inoltre la raccolta delle metriche sperimentali necessarie per confrontare il comportamento dei diversi algoritmi.

I dati del grafo sono organizzati a partire da due sorgenti principali: una contenente i nodi e una contenente gli archi. Durante il caricamento, a ogni nodo viene associato un **identificativo** e una **label** testuale. Per rendere più flessibile l'esecuzione, vengono costruite due strutture di supporto: una che permette di risalire dalla label all'ID del nodo e una che consente l'operazione inversa. Gli archi vengono invece utilizzati per costruire una **lista di adiacenza**, cioè una struttura che associa a ogni nodo l'elenco dei suoi vicini.

La lista di adiacenza rappresenta il grafo in memoria e permette agli algoritmi di accedere rapidamente ai successori di ogni nodo. Nel caso in cui il grafo venga considerato non diretto, per ogni collegamento viene aggiunto automaticamente anche l'arco inverso. Prima dell'inserimento, il programma verifica che entrambi i nodi coinvolti nell'arco siano presenti nel dataset, evitando quindi collegamenti non validi.

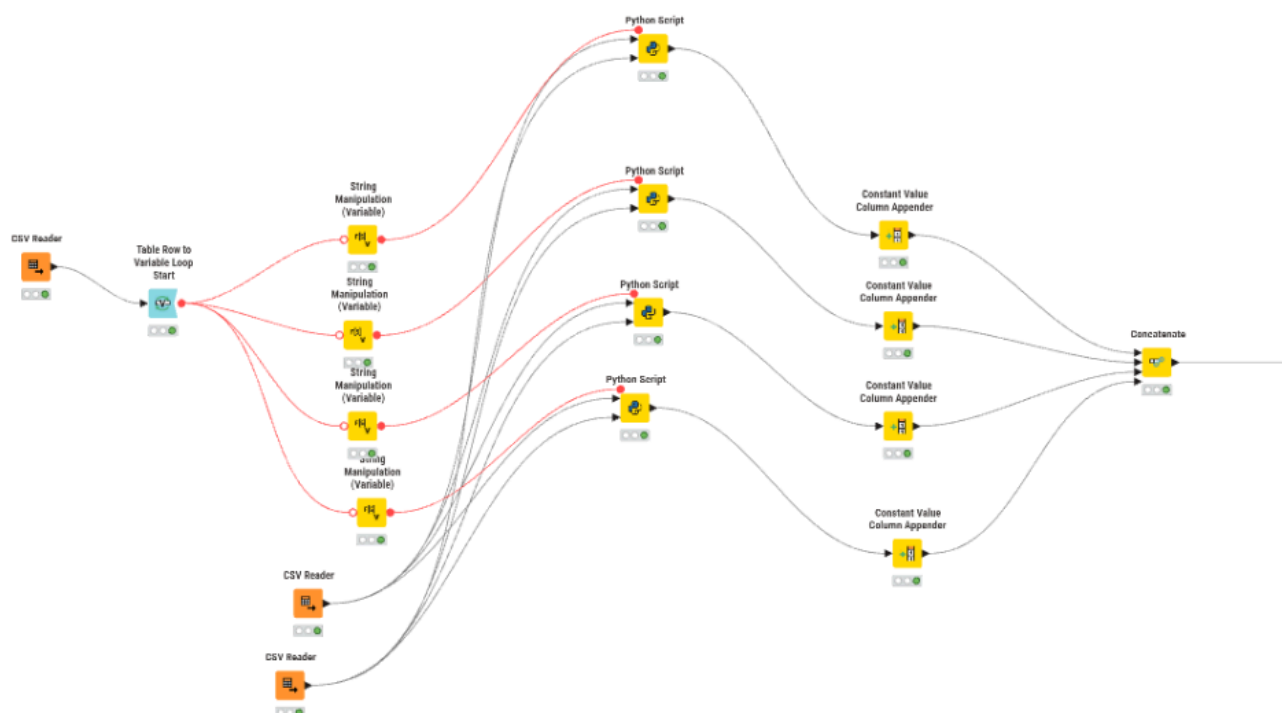
L'esecuzione della ricerca parte da un nodo iniziale e termina quando viene raggiunto il nodo obiettivo oppure quando non esistono più stati da esplorare. La BFS utilizza una **coda FIFO** e visita il grafo per livelli; la DFS utilizza una **pila LIFO** e procede in profondità; la DLS limita l'esplorazione a una profondità massima stabilita; la IDS esegue più ricerche DLS aumentando progressivamente il limite.

Durante l'esecuzione vengono registrate alcune metriche fondamentali: il numero di iterazioni effettuate, il tempo di esecuzione in millisecondi, la lunghezza del percorso trovato e il numero massimo di stati mantenuti nelle strutture dati ausiliarie. Se il nodo obiettivo viene raggiunto, il percorso viene ricostruito tramite il **dizionario dei predecessori** e salvato come sequenza ordinata di nodi.

L'output prodotto contiene le informazioni necessarie per l'analisi successiva: algoritmo utilizzato, esito della ricerca, lunghezza del percorso, percorso individuato, iterazioni, memoria stimata tramite gli stati memorizzati, tempo di esecuzione e messaggio finale. Questo formato consente a KNIME di importare, aggregare e visualizzare facilmente i risultati ottenuti.

6. Workflow KNIME

Il workflow KNIME è stato progettato per automatizzare l'esecuzione degli esperimenti, gestire più coppie di nodi in modalità batch e confrontare i risultati ottenuti dagli algoritmi di ricerca. In questa fase, KNIME non implementa direttamente la logica degli algoritmi, ma coordina il caricamento dei dati, l'esecuzione degli **script Python**, l'aggregazione dei risultati e la produzione dei grafici finali.



La prima parte del workflow riguarda il caricamento degli input. Un nodo CSV Reader importa il file contenente le coppie di nodi da testare. Ogni riga rappresenta un esperimento composto da un nodo iniziale e da un nodo obiettivo. Il nodo **Table Row to Variable Loop Start** trasforma ogni riga della tabella in variabili di flusso, permettendo al workflow di eseguire automaticamente una serie di test, uno per ogni coppia presente nel file.

All'interno del ciclo sono presenti quattro nodi **String Manipulation** (Variable), utilizzati per preparare i parametri relativi ai quattro algoritmi considerati: *BFS*, *DFS*, *IDS* e *DLS*. Queste variabili vengono poi passate ai rispettivi nodi Python Script, che rappresentano il cuore computazionale del workflow. Sono infatti presenti quattro nodi Python distinti, uno per ciascun algoritmo.

I file relativi ai nodi e agli archi del grafo vengono caricati tramite ulteriori nodi CSV Reader e forniti in input ai nodi Python Script. I nodi Python elaborano quindi il grafo, eseguono la ricerca e restituiscono le metriche calcolate.

Dopo l'esecuzione, ogni ramo del workflow passa attraverso un nodo **Constant Value Column Appender**, che aggiunge una colonna costante per identificare l'algoritmo di riferimento. Questo passaggio consente di distinguere chiaramente i risultati prodotti da BFS, DFS, IDS e DLS dopo l'unione dei dati.

I quattro flussi vengono successivamente raccolti dal nodo **Concatenate**, che combina verticalmente le tabelle prodotte dai diversi algoritmi. Il nodo **Loop End** chiude il ciclo batch e restituisce una tabella complessiva contenente tutti i risultati ottenuti per tutte le coppie di nodi analizzate.



Nella seconda parte del workflow vengono effettuate le operazioni di pulizia, filtro e aggregazione. Il nodo **Rule Engine** viene utilizzato per trasformare o normalizzare alcuni valori, ad esempio convertendo l'esito della ricerca in una forma più adatta all'analisi statistica. Il nodo **Row Filter** permette invece di selezionare solo le righe utili alle elaborazioni successive. I nodi **GroupBy** calcolano valori aggregati per algoritmo, come tempo medio, numero medio di iterazioni, lunghezza media del percorso e percentuale di successo.

I risultati aggregati vengono poi combinati tramite un nodo **Joiner**, rinominati con **Column Renamer** e ordinati tramite **Sorter**, così da ottenere una tabella finale più leggibile. Il nodo **Math Formula** consente di calcolare ulteriori indicatori, come la percentuale di successo espressa in forma percentuale.

Infine, il workflow produce sia un file CSV finale tramite **CSV Writer**, sia una serie di visualizzazioni grafiche. In particolare, vengono generati grafici a barre e grafici lineari, utili per confrontare visivamente gli algoritmi rispetto alle metriche principali. In questo modo KNIME svolge il ruolo di ambiente di automazione, analisi e reportistica, mentre l'elaborazione algoritmica viene affidata ai nodi Python Script.

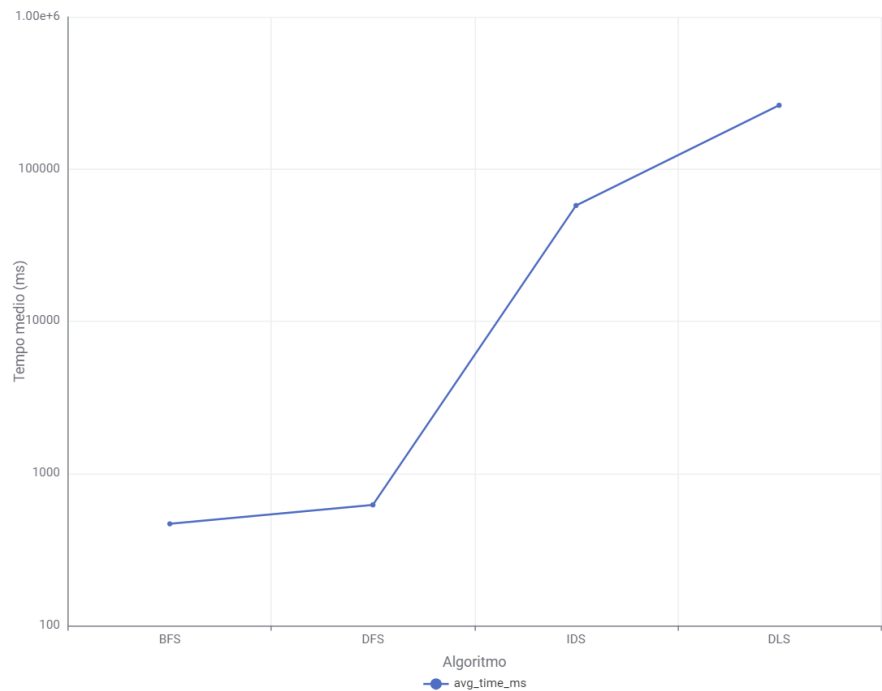
7. Analisi dei Risultati Sperimentali e Reportistica Grafica

I dati raccolti mediante l'esecuzione batch del workflow KNIME permettono di confrontare il comportamento pratico degli algoritmi BFS, DFS, DLS e IDS rispetto alle principali metriche sperimentali: tempo di esecuzione, numero di iterazioni, lunghezza del percorso e stati massimi memorizzati.

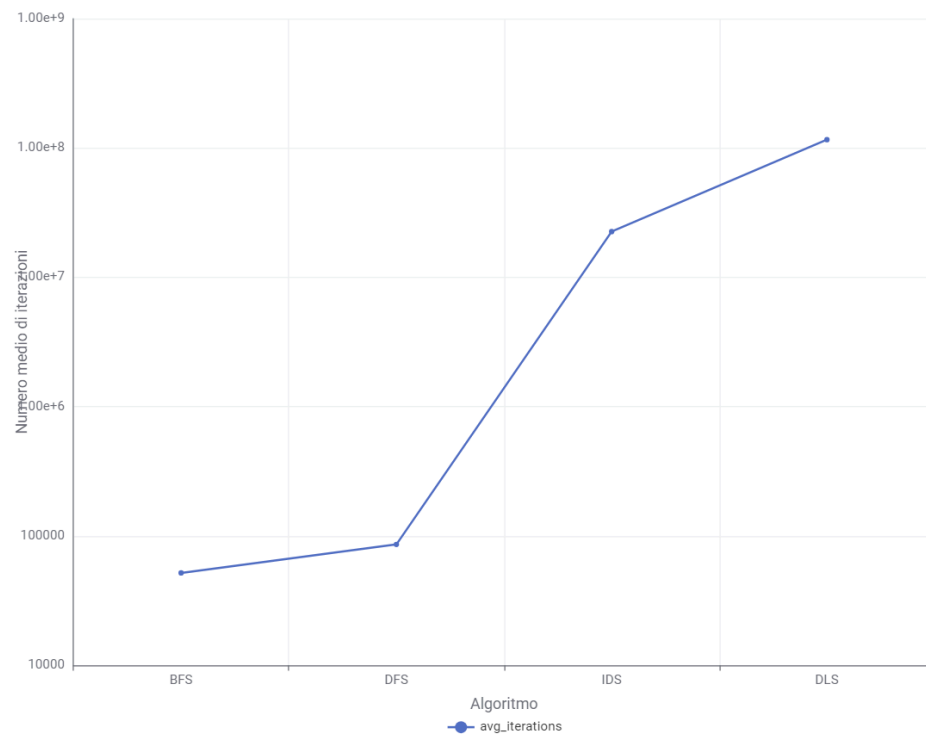
Dai grafici prodotti emerge che ogni algoritmo presenta vantaggi e limiti differenti. La BFS risulta particolarmente indicata quando l'obiettivo principale è individuare il percorso minimo in un grafo non pesato, ma può richiedere una maggiore quantità di memoria. La DFS, invece, tende a utilizzare meno memoria, ma il percorso trovato dipende fortemente dall'ordine di visita dei nodi e non sempre risulta ottimale.

La DLS consente di limitare la profondità massima della ricerca, evitando esplorazioni troppo estese. Tuttavia, se il limite scelto è inferiore alla profondità effettiva del goal, l'algoritmo non riesce a individuare una soluzione. La IDS rappresenta un buon compromesso, perché combina l'esplorazione progressiva per profondità con un consumo di memoria più contenuto rispetto alla BFS. Nel complesso, l'analisi grafica conferma quanto discusso nei capitoli teorici: non esiste un algoritmo migliore in assoluto, ma la scelta dipende dal tipo di grafo, dalla profondità della soluzione e dai vincoli di tempo e memoria disponibili.

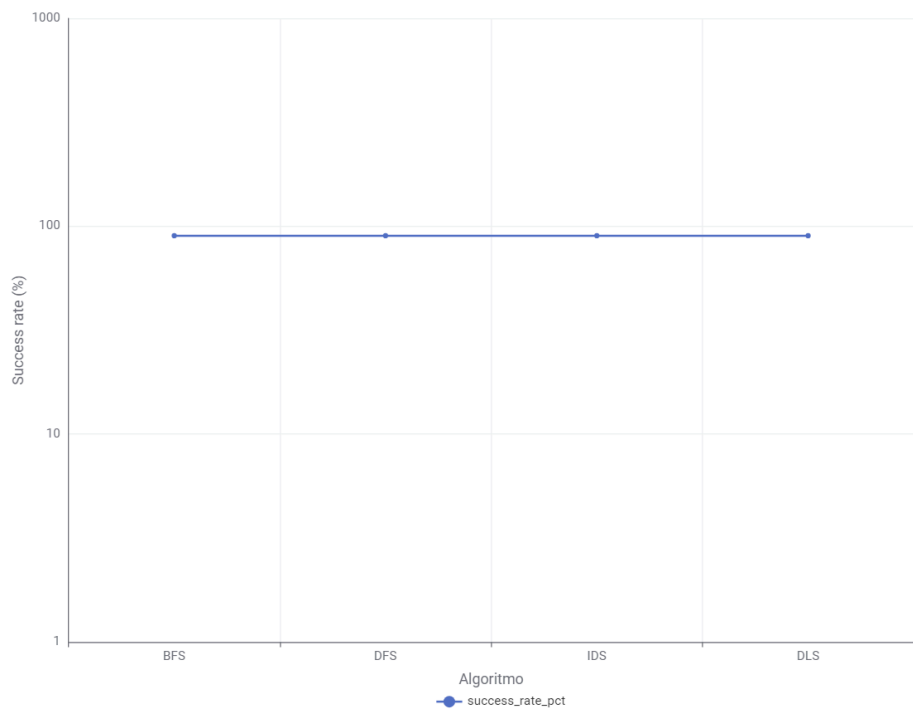
Tempo medio per algoritmo



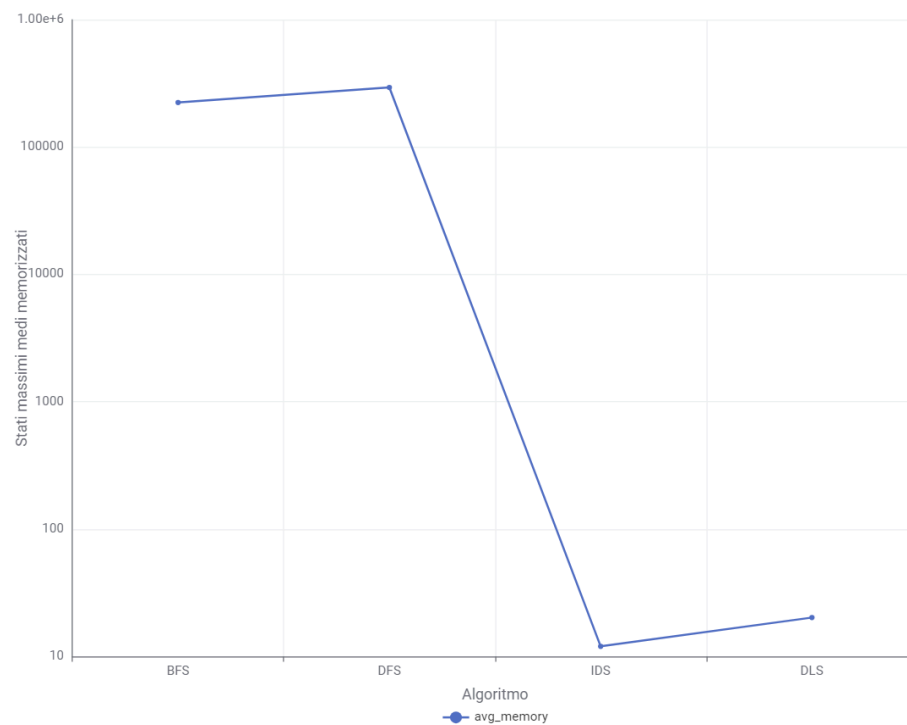
Iterazioni medie per algoritmo



Percentuale di successo per algoritmo



Stati massimi medi memorizzati



8. Conclusioni e Linee Guida per la Scelta dell'Algoritmo Ottimale

In ultima analisi, la ricerca sperimentale condotta tramite script Python e workflow KNIME permette di definire alcune linee guida per la scelta dell'algoritmo più adatto in base ai vincoli del problema e alle caratteristiche del grafo analizzato.

- La **Breadth-First Search (BFS)** è indicata quando l'obiettivo principale è trovare il percorso minimo in un grafo non pesato. Tuttavia, può richiedere un consumo di memoria elevato, soprattutto quando il numero di nodi cresce rapidamente.
- La **Depth-First Search (DFS)** utilizza meno memoria rispetto alla BFS, ma non garantisce sempre il percorso ottimale e può avere un comportamento meno prevedibile, specialmente in presenza di grafi profondi o con molti cicli.
- La **Depth-Limited Search (DLS)** permette di controllare la profondità massima dell'esplorazione, evitando ricerche eccessivamente estese. Tuttavia, se il limite scelto è inferiore alla profondità del goal, l'algoritmo non riesce a trovare la soluzione.
- L'**Iterative Deepening Search (IDS)** rappresenta un buon compromesso tra BFS e DFS. Essa combina la capacità di trovare soluzioni ottimali nei grafi non pesati con un consumo di memoria più contenuto, risultando particolarmente adatta quando non si conosce in anticipo la profondità della soluzione.